

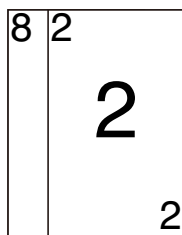
Selection Sort

选择排序

本页面将简要介绍选择排序。

定义

选择排序（英语：Selection sort）是一种简单直观的排序算法。它的工作原理是每次找出第 i 小的元素（也就是 $A[i..n]$ 中最小的元素），然后将这个元素与数组第 i 个位置上的元素交换。



性质

稳定性

选择排序的稳定性取决于其具体实现。

倘若使用链表实现，由于链表的任意位置插入和删除均为 $O(1)$ ，故无需使用 swap（交换两个元素）操作：每次从未排序部分选择最小元素（若有多个，选取第 1 个）后，将其插入到未排序部分的第 1 个元素之前，这样就能够保证稳定性。

假如使用数组实现（OI 中一般的实现方式），由于数组任意位置插入和删除均为 $O(n)$ ，故只能使用 swap 将未排序部分的元素移到已排序部分。swap 操作使得数组实现的选择排序不稳定。

下面给出的实现示例均是基于数组元素的交换，因此均为 **不稳定的**。

时间复杂度

选择排序的最优时间复杂度、平均时间复杂度和最坏时间复杂度均为 $O(n^2)$ 。

代码实现

伪代码



C++PythonJava

```
1 #include <utility>
2
3 void selection_sort(int* a, int n) {
4     for (int i = 1; i < n; ++i) {
5         int ith = i;
6         for (int j = i + 1; j <= n; ++j) {
7             if (a[j] < a[ith]) {
8                 ith = j;
9             }
10        }
11        std::swap(a[i], a[ith]);
12    }
13 }

1 def selection_sort(a, n):
2     for i in range(1, n):
3         ith = i
4         for j in range(i + 1, n + 1):
5             if a[j] < a[ith]:
6                 ith = j
7         a[i], a[ith] = a[ith], a[i]

1 // arr代码下标从 1 开始索引
2 static void selection_sort(int[] arr, int n) {
3     for (int i = 1; i < n; i++) {
4         int ith = i;
5         for (int j = i + 1; j <= n; j++) {
6             if (arr[j] < arr[ith]) {
7                 ith = j;
8             }
9         }
10        // swap
11        int temp = arr[i];
12        arr[i] = arr[ith];
13        arr[ith] = temp;
14    }
15 }
```

本页面最近更新：2025/1/1 23:29:33, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [NachtgeistW](#), [Early0v0](#), [Enter-tainer](#), [iamtwz](#), [Junyan721113](#), [ksyx](#), [mcendu](#), [Menci](#), [ouuan](#), [partychicken](#), [shawlleyw](#), [tfia](#), [Tiphereth-A](#), [TrickEye](#), [WException](#), [Xeonacid](#), [ZbohZbp](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用